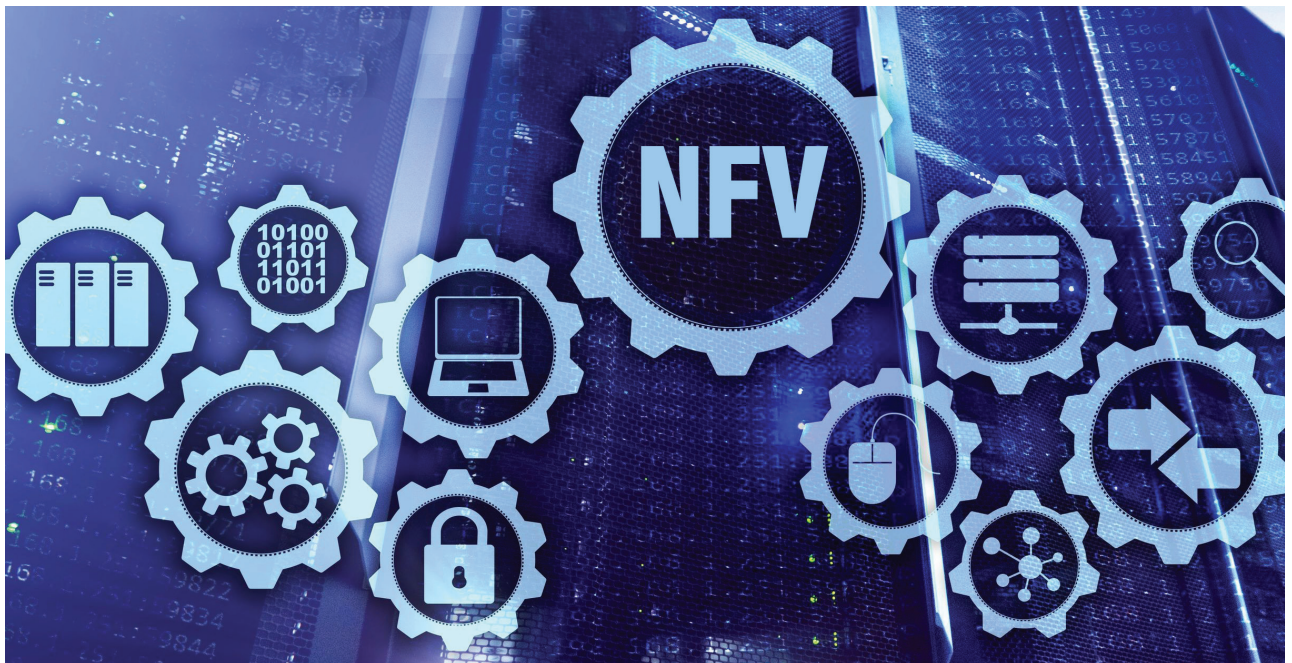


Integrating Network Function Virtualization with the DevOps Pipeline: Containerization

This multi-part series of articles will take you through the various components of DevOps. It will introduce you to network function virtualization and its integration with the DevOps pipeline. This second article in the series focuses on containerization.



Virtualization is today the backbone of many data centers around the globe, providing resources for modern-day applications. Containerization is a sort of virtualization, where multiple user spaces can be created over the top of the existing kernel. We can define a container as a plug-and-play box, which contains all the dependencies required for the application or service inside of it to run.

Containers are well known for less overhead and are more performant compared to hypervisor-based virtualization. When VMs are compared

with containers, Docker containers have outperformed KVMs (kernel-based virtual machines) in overhead for CPU, memory performance, and I/O-intensive workload. Containers are used to ship software, applications, or services. For example, if you want to set up your web application on a web server, you can do it via a portable tool like a container. You put all the required dependencies and the application artifact in the container of a web server and run it for your application to be available to make any request. Figure 1 depicts the hypervisor and container architecture.

Architecture

Advances in the Linux kernel made OS virtualization possible. Now the kernel provides mechanisms such as name spaces and control groups. With these advances, one can isolate processes on the host machine. This is supported via the Linux container (LXC) project.

Let us discuss in brief the internals of the container. A set of processes is called a group. For each group, we can set process attributes and a proportionate share of system resources like CPU, memory, disk space, and